Ejercicios del Tema 3

Tras haber leído y comprendido el tema 3, resuelve estos ejercicios.

Ejercicio 1 – Herencia para añadir funcionalidad

Continuando el ejercicio de la clase `IntArrayStack` se pide:

- Heredar de dicha clase para crear la clase `IntArrayStackPlus`. La clase `StackPlus` incorpora el método `size` y el método `empty`.
- Probar a insertar y retirar diferentes enteros del `IntArrayStack` y comprobar el tamaño.
- Hacer debug para comprobar cómo se invoca a la clase derivada y luego a la clase padre.

Ejercicio 2 – Herencia para restringir funcionalidad

Continuando el ejercicio de la clase `IntArrayStack` se pide:

- Heredar de dicha clase para crear la clase `PairIntArrayStack`. Dicha clase sobreescribe el método `push` para que solo permita insertar enteros pares.
- Probar a insertar y retirar diferentes enteros pares e impares y comprobar el resultado.
- Piensa en las ventajas de que la clase `IntArrayStack` fuese `final`.

Ejercicio 3 – Polimorfismo para operaciones generales

Crear una clase `Cleaner` que tenga un único método `clear` que sea capaz de vaciar la estructura `IntArrayStack` o cualquier clase que derive de ella (como `PairIntArrayStack`). Prueba la clase `Cleaner` insertando varios elementos en un `IntArrayStack` y luego llamando a `clear` para borrarlos.

Ejercicio 4 – Polimorfismo para generalizar estructuras datos

Basándonos en la implementación de la clase `IntArrayStack` se pide crear la clase `ArrayStack` que permita insertar elementos de tipo `Object`.

- Sobre dicha clase, prueba a insertar diferentes tipos de objetos (`Coche, String...`).
- Piensa en el problema de hacer `push` y meter un objeto de tipo `Coche` y luego hacer `pop` y que devuelva un `Coche`.

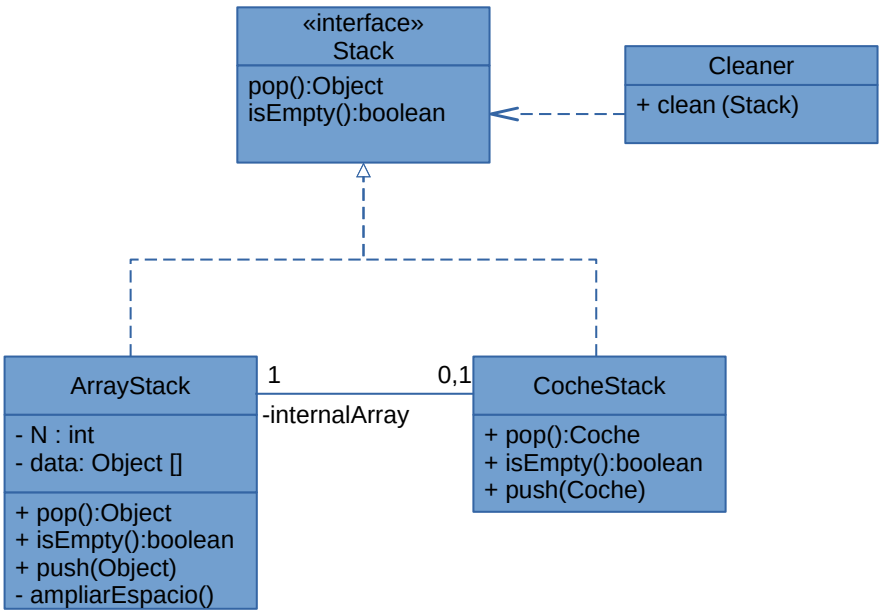
Ejercicio 5 – Polimorfismo y estructuras de dato seguras

Intenta crear una clase llamada `CocheArrayStack` que herede de `ArrayStack` y que solo permita insertar objetos de la clase `Coche`. ¿Es posible crear dicha clase?

Ejercicio 6 – Estructuras de datos más seguras

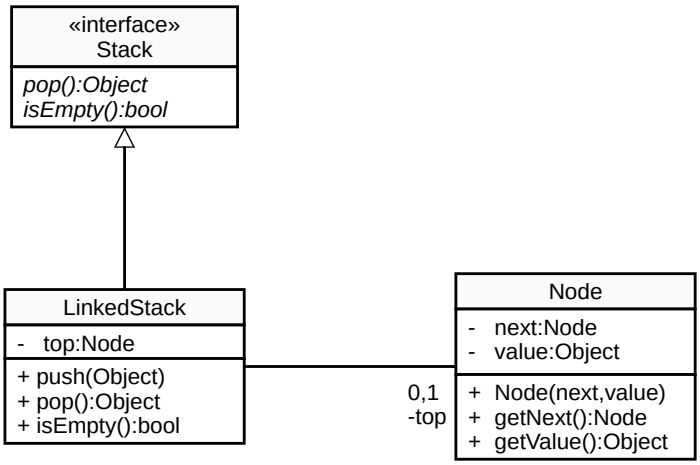
Siguiendo el esquema de la figura adjunta se pide:

- Crear una interfaz llamada `Stack` y hacer que `ArrayStack` la implemente.
- Discutir las ventajas de la estructura de clases del diagrama adjunto.
- ¿Explicar por qué no se define el método `push` en la interfaz `Stack`?
- Modificar `Cleaner` para que funcione con `Stack`.



Ejercicio 7 – Varias implementaciones que comparten interfaz

Crear la clase `LinkedStack` que implemente la interfaz `Stack` usando nodos enlazados. La siguiente figura ilustra la implementación que se pide.



Ejercicio 8 – Jerarquía de operaciones similares

Con objeto de crear una jerarquía de clases (ver figura adjunta) que nos permitan realizar operaciones sobre los arrays de caracteres, se pide:

- Crear una interfaz llamada `CharArrayOperation` con un único método llamado `doOperation` que recibe un array de `char`.
- Crear las siguientes clases que implementan dicha interfaz y crear ejemplos de uso de cada una de ellas:
  - `Upper`: los objetos de esta clase pasan a mayúsculas todos los caracteres del array recibido.
  - `Lower`: los objetos de esta clase pasan a minúsculas todos los caracteres del array recibido.
  - `CapitalizeWords`: los objetos de esta clase ponen en mayúsculas la primera letra de el array y la priemra letra tras los caracteres de espacio y de puntuación (punto, coma…).

- `Reverse`: un objeto `Reverse`, invierte los caracteres del array que se le pasa. Por ejemplo, “juan” lo convierte en “nauj”.
-

